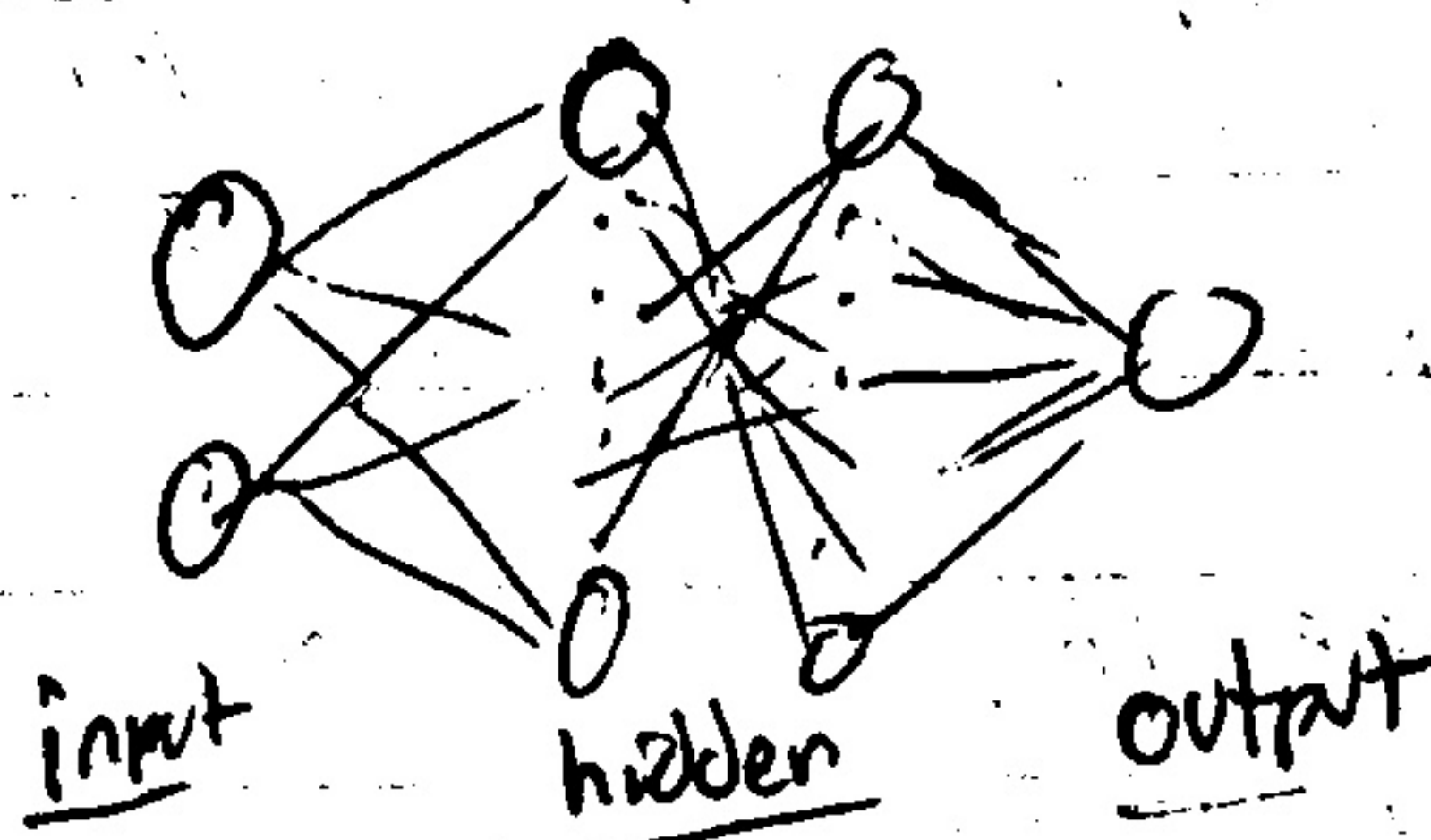


• Loss function: $MSE = L(\theta) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$ where $\hat{y}_i = f_{\theta}(x_i)$ is the prediction

• MLP: function $f_{\theta}(x_i)$

- where x_i is one input point
- θ is our parameters, every weight and bias in the MLP
- $f_{\theta}(x_i)$ is the output produced by the MLP
- $\hat{y}_i$ is the prediction
- architecture: 2 inputs, 2 hidden-layers (16 neurons), 1 output



make-moons sample:

$$x_i = \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix} \quad y_i = 1 \text{ or } -1$$

• Structure of data:

- X has shape 100×2 (100, 2)
- Y has shape (100,) contains 100 ~~and~~ labels
- One training pair: $(x_i, y_i) = \left(\begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}, y_i \right)$

• Training: $\theta = [w_1, w_2, \dots, b_1, b_2, \dots]^T$ Parameters

At each epoch k , network has parameters θ_k and loss

$$L(\theta_k)$$

1. Backward Pass

calculate the gradient

$$\text{loss.backward}() \Rightarrow g_k = \nabla L(\theta_k) \text{ where } \nabla L(\theta_k) = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} \\ \vdots \\ \frac{\partial L}{\partial \theta_n} \end{bmatrix}$$

The gradient points in the direction of greatest local increase in loss. Therefore, steepest descent moves in opposite direction.

$$d_k = -g_k$$

2. Learning Rate (Gradient Descent):

- Update: $\theta_{k+1} = \theta_k - \alpha_k g_k$

$\{ -g_k \}$ is the direction

$\{ -\alpha_k \}$ is the learning rate

$\alpha_k \|g_k\|$ is the actual distance moved in parameter space

Micrograd

3. Use Line-Search to transform into one-dimensional problem

Once gradient direction is fixed, only search over scalar α

• Define: $\phi_k = L(\theta_k - \alpha g_k)$ ← what would the loss be if we moved α units along descent direction

4. Armijo Condition

Accept α when:

Armijo constant ($c=10^{-4}$)

$$L(\theta_k - \alpha g_k) \leq L(\theta_k) - c \alpha \|g_k\|^2$$

$$\|g_k\|^2 = g_k^T g_k = \sum_{i=1}^n \left(\frac{\partial L}{\partial \theta_i} \right)^2$$

Left side:

Right side: Maximum acceptable loss

$$L_{\text{candidate}} = L(\theta_k - \alpha g_k) \quad L_{\text{required}} = L(\theta_k) - c \alpha \|g_k\|^2$$

- ~~Loss~~ • If new loss is unacceptable, shrink learning rate by shrink factor, start learning rate loop again
- Otherwise, return new learning rate

5. Permanently update the parameters